

CPU Execution Lab

Presented by Rafan Ahmed
CAIR Blackhole Project, September 15, 2026

Itinerary for the Week

- Confirm that everyone has the tools needed for the lab
- Build a simple model of CPU execution and memory
- Preview the experiments that will become the shared benchmark
- Connect the CPU observations to machine learning and Blackhole

Setup checklist

- Your fork exists and its blank lab structure opens in VS Code and Visual Studio Code opens normally
- Git is installed and prints a version
- A terminal opens and accepts commands
- At least one C compiler prints a version

Opening your cloned fork in VS Code

- Open the [Visual Studio Code](#) application
- Select [File > Open Folder](#)
- Find the location where you cloned your fork
- Select the main [blackhole](#) folder, then open it
- Confirm that README.md appears
- Select Terminal > New Terminal, then run `git --version`

Keeping your fork current

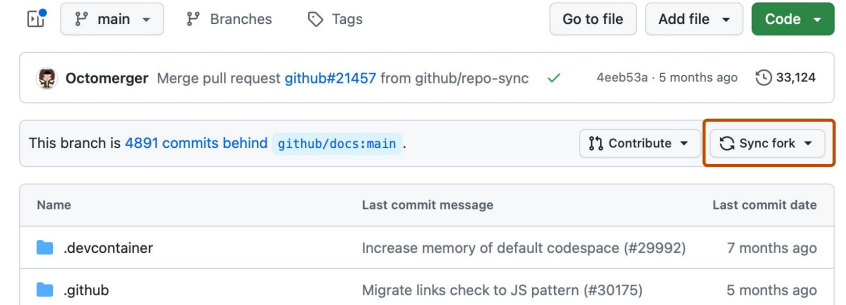
1. Update the fork on GitHub website

- Open your fork
- Select “Sync fork”
- Select “Update branch”

2. Update the local clone in the VS Code terminal

```
git switch main  
git pull origin main
```

A plain git pull usually checks your fork. New upstream files appear after the fork has been synchronized.



Example from
<https://docs.github.com/en/pull-requests/how-to-work-with-forks/syncing-a-fork>

Compiler setup

macOS

```
xcode-select --install
```

- Installs Apple Clang and the cc compiler command.

Linux (Ubuntu or Debian)

```
sudo apt update
```

```
sudo apt install git build-essential
```

- Installs GCC, development headers, and standard build tools.

Verify after installation

```
cc --version
```

Windows compiler setup with WSL

Open PowerShell as Administrator

```
wsl --install
```

Restart your computer, then open Ubuntu

Start menu > search for Ubuntu > open the Ubuntu application

Complete Ubuntu's first-time setup

Create a Linux username and password

Your password stays invisible while you type. This is normal.

Install the lab tools inside Ubuntu

```
sudo apt update  
sudo apt install git build-essential
```

Verify that the compiler works

```
cc --version
```

Build and run preview

After you sync and pull the starter code update, compile from the lab folder:

```
cc -std=c11 -O2 -Wall -Wextra -pedantic -Iinclude src/*.c -o cpu_lab
```

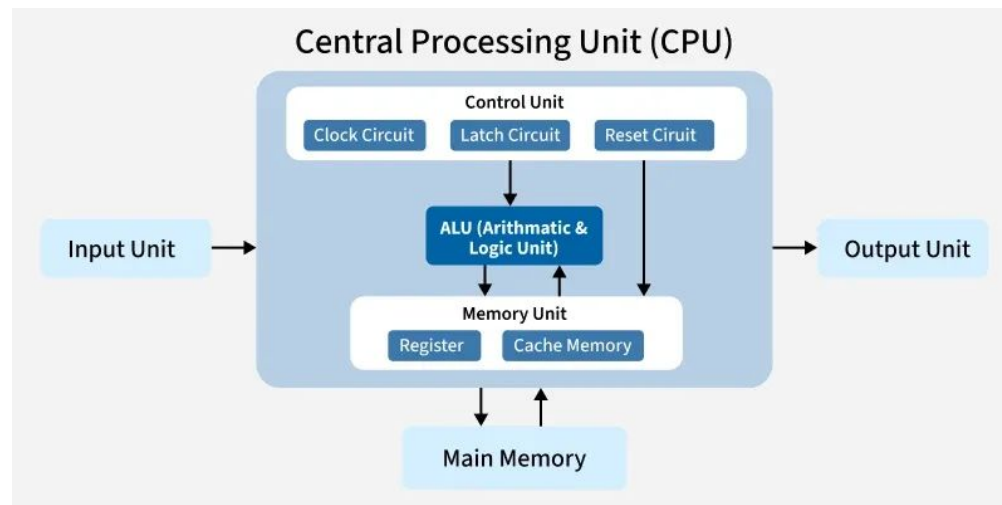
Run the executable:

```
./cpu_lab
```

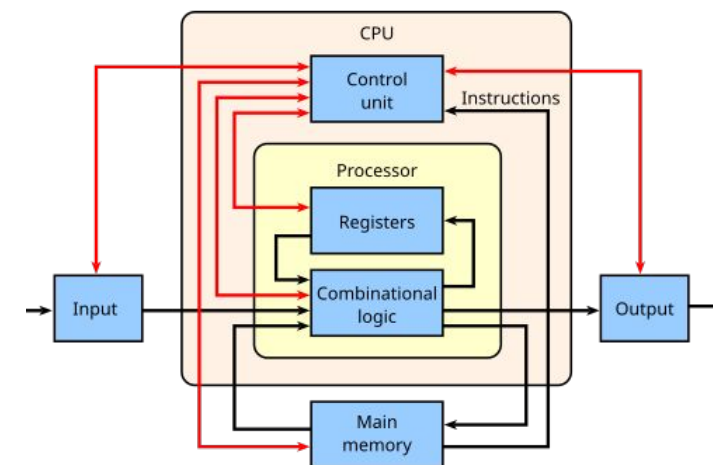
The terminal will print results from each benchmark experiment.

The CPU

- The CPU is the computer's general purpose processor
- It executes streams of machine instructions
- Registers hold values currently used by instructions
- Arithmetic units perform operations such as addition and multiplication
- Caches keep recently used data close to the processor



<https://www.geeksforgeeks.org/computer-science-fundamentals/central-processing-unit-cpu/>

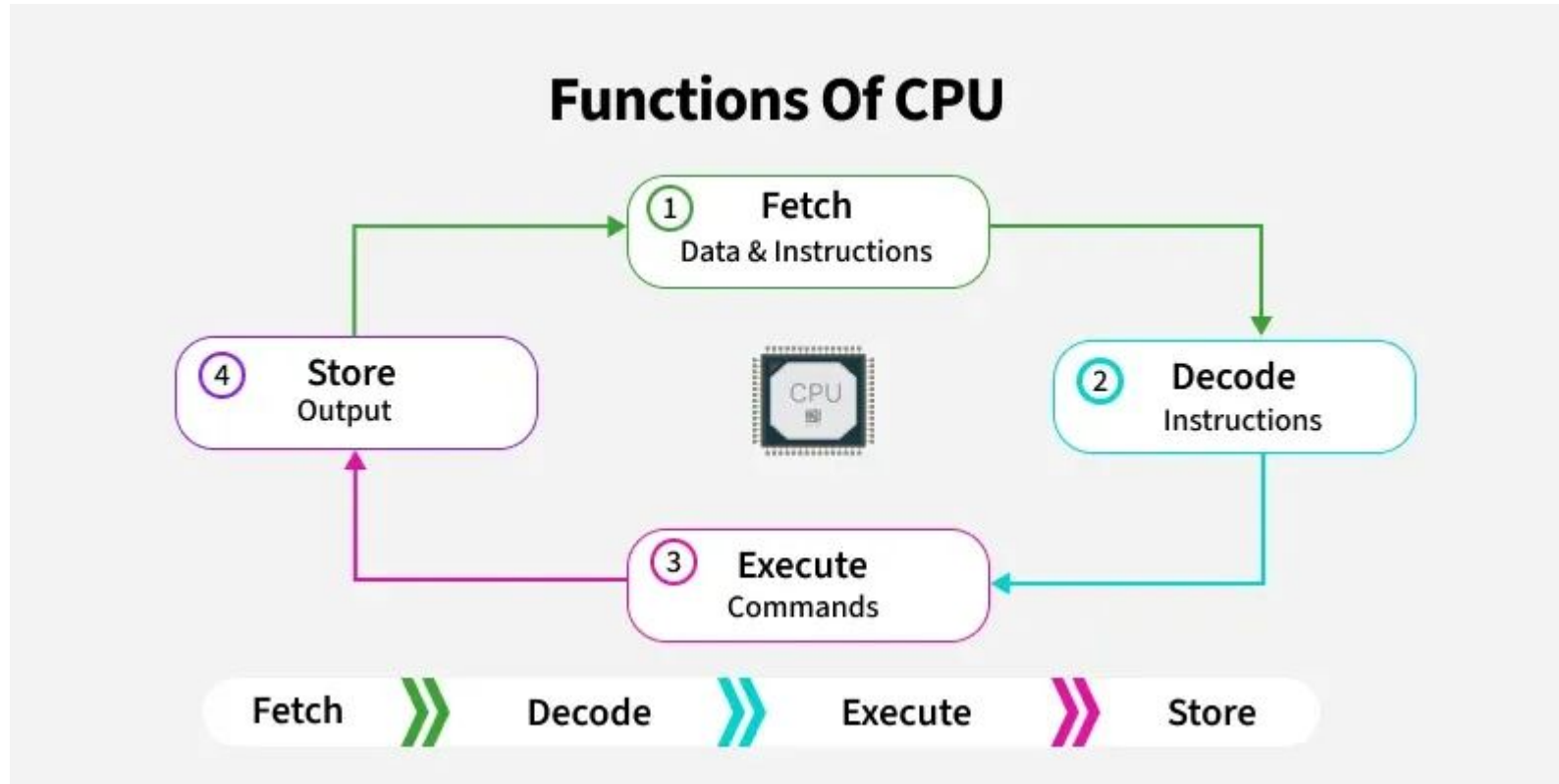


https://en.wikipedia.org/wiki/Central_processing_unit

From C source to a running process

1. You write a C source file.
2. The compiler creates machine code and an executable.
3. The operating system starts a process and maps its code and data into memory.
4. The CPU fetches and executes the program's instructions.

Functions of CPU:



- **Fetch:** The CPU retrieves the instruction from main memory (RAM).
- **Decode:** The Control Unit interprets the fetched instruction to determine the required operation.
- **Execute:** The CPU performs the operation using the appropriate hardware components such as the ALU.
- **Store:** The result of the executed instruction is written back to memory or a register.

Text description and image are both from <https://www.geeksforgeeks.org/computer-science-fundamentals/central-processing-unit-cpu/>

Machine Instructions

- **Load:** bring a value from memory into a register
- **Store:** write a value from a register back to memory
- **Arithmetic:** add, multiply, or update an address
- **Compare and branch:** decide whether a loop continues
- **Registers:** small named locations used directly by the CPU

Main Parts of a Machine Instruction

- **Opcode (Operation Code):** Tells the processor what action to perform, such as add, subtract, or move data.
- **Operand:** Specifies the data or the memory address where the data is located.
- **Addressing Mode:** Explains how the CPU can find or access the operand

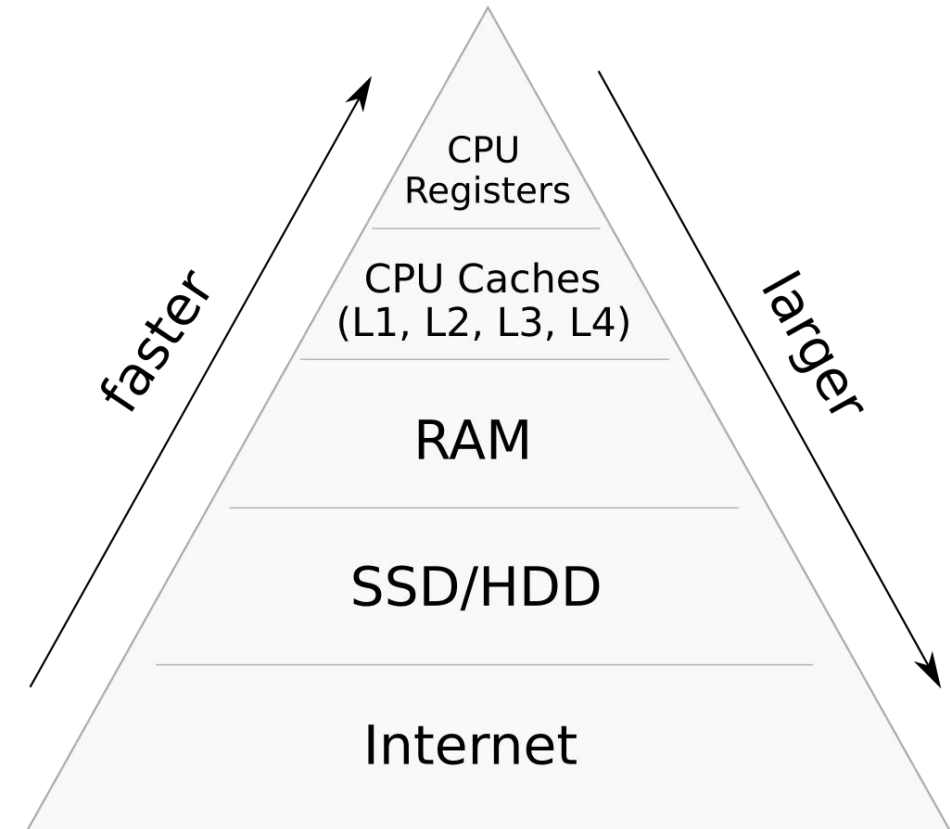
Memory hierarchy

Registers A few values used directly by current instructions

Caches L1, L2, and often L3 keep nearby or recently used data

DRAM Larger working memory for running programs

Accessing data from farther levels usually costs more time!!



From
<https://en.algorithmica.org/hpc/external-memory/hierarchy/>

The lab question

How much can performance change when the computation stays the same but the data moves differently?

We will change access order, layout, and work division. Then we will measure the result.

Memory access experiments

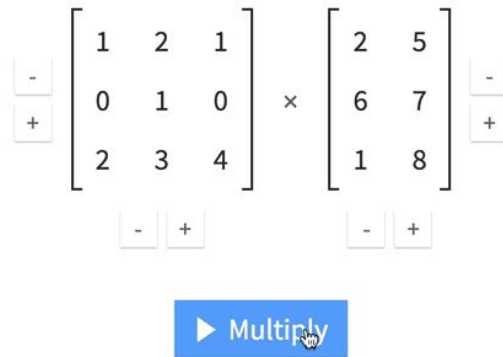
- **Strided access:** touch the same array with different gaps between elements
- **Row and column traversal:** visit the same matrix in two orders
- **Bandwidth copy:** measure the cost of moving a large block of data
- The total data can stay similar while the access pattern changes

Benchmarking

- Check that every implementation produces the expected result
- Test several input sizes
- Run each case more than once
- Use the same compiler settings for comparisons
- Compare patterns on one machine before comparing different machines
- Record unusual conditions such as heavy background activity

Matrix multiplication

Matrix Multiplication



An interactive interface for matrix multiplication. It shows two 3x3 matrices being multiplied. The first matrix has values 1, 2, 1 in the first row; 0, 1, 0 in the second row; and 2, 3, 4 in the third row. The second matrix has values 2, 5 in the first row; 6, 7 in the second row; and 1, 8 in the third row. Each matrix is flanked by minus and plus signs. Below the matrices is a blue button with a play icon and the text 'Multiply'.

$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 0 \\ 2 & 3 & 4 \end{bmatrix} \times \begin{bmatrix} 2 & 5 \\ 6 & 7 \\ 1 & 8 \end{bmatrix}$$

▶ Multiply

How to Multiply 2 × 2 Matrices

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{bmatrix}$$

From

<https://datachant.com/2016/06/01/excel-matrix-multiplication-replacing-mmult-with-power-query/>

<https://www.geeksforgeeks.org/maths/how-to-multiply-2-x-2-matrices/>

<https://ru.pinterest.com/pin/477029785519648105/>

$$(\mathbf{AB})_{ij} = \sum_{k=1}^m A_{ik} B_{kj}$$

- Each output value is a dot product
- The inner loop repeats multiply and add operations
- Many machine learning layers contain matrix multiplication
- A useful shorthand is input multiplied by weights

Naive and transposed matmul

Naive access

$A[i][k]$ reads across a row
 $B[k][j]$ jumps between rows

Transposed access

$A[i][k]$ reads across a row
 $B^T[j][k]$ also reads across a row

Even though mathematical result stays the same, the memory access pattern changes!

Measure transpose cost separately or state clearly when the benchmark excludes it.

Assembly view

```
cc -std=c11 -O2 -Iinclude -S src/stride_access.c -o stride_access.s
```

- Register names
- Load and store instructions
- Arithmetic instructions
- Compare and branch instructions
- Repeated instruction groups that form loops

Optional threading experiment

- Divide a large array into separate ranges
- Let each thread compute one partial sum
- Combine the partial sums after all threads finish
- Thread creation and coordination also consume time
- Parallel work helps when the task is large enough and divides cleanly

Why AI accelerators exist

- Machine learning workloads repeat large amounts of regular numerical work
- Specialized matrix and vector engines can process that work efficiently
- Local memory can keep reused data close to compute
- Many workers can process independent pieces of a larger tensor
- A fast interconnect and memory system must keep those workers supplied

After the lecture

- Wait for the announcement that the starter code commit is ready
- Open your GitHub fork and use Sync fork
- In the local clone, switch to main and pull from origin
- Follow `cpu-execution-lab.md` for the full setup and lab guide
- Confirm that the lab builds and runs from the terminal
- Create a work branch, complete the full lab, and open an individual pull request

Blackhole preview

- RISC-V processors coordinate device work
- Local L1 SRAM keeps working data near compute
- The Network-on-Chip moves data across the device
- Specialized matrix and vector engines operate on tensor data
- GDDR6 stores larger data on the device